

Sentence Memorability Graphs

```
raw_dataset <- read_csv(
  "sentence_memorability_data_h8.csv",
  na = c("", "NA", "N/A"),
  show_col_types = FALSE
)

blocked_aggregated <- raw_dataset %>%
  group_by(participant_ID, Block_Number) %>%
  mutate(
    Accuracy.IR = suppressWarnings(as.numeric(Accuracy.IR)),
    Accuracy.WR = suppressWarnings(as.numeric(Accuracy.WR)),
    prev_IR = lag(Accuracy.IR)
  ) %>%
  summarise(
    # IR components
    Total_Repeats = sum(isRepeat == TRUE & Event == "Sentence shown", na.rm = TRUE),
    Total_Denom = sum(is.na(isRepeat) & Event == "Sentence shown", na.rm = TRUE),
    Hits = sum(Event == "IR pressed" & isRepeat == TRUE & Accuracy.IR == 1, na.rm = TRUE),
    False_Alarms = sum(Event == "IR pressed" & is.na(isRepeat), na.rm = TRUE),

    # IR metrics
    Hit_Rate = if_else(Total_Repeats > 0, Hits / Total_Repeats, 0),
    False_Alarm_Rate = if_else(Total_Denom > 0, False_Alarms / Total_Denom, 0),
    IR_Memorability = Hit_Rate - False_Alarm_Rate,

    # WR components
    WR_correct = sum(Event == "WR pressed" & Accuracy.WR == 1 & prev_IR == 1, na.rm = TRUE),
    WR_total = sum(Event == "WR pressed" & prev_IR == 1, na.rm = TRUE),

    # WR metric
    WR_Accuracy = if_else(WR_total > 0, WR_correct / WR_total, 0),
    .groups = "drop"
  ) %>%
  # Ensure the Block_Number is a categorical factor for plotting/testing
  mutate(Block_Number = factor(Block_Number, levels = c(1, 2, 3), labels = c("Block 1", "Block 2", "Block 3")))

print(paste(sum(blocked_aggregated$IR_Memorability > 1 | blocked_aggregated$IR_Memorability < 0, na.rm = TRUE),
  collapse = ", "))

## [1] "0"

print(paste(sum(blocked_aggregated$WR_Accuracy > 1 | blocked_aggregated$WR_Accuracy < 0, na.rm = TRUE),
  collapse = ", "))

## [1] "0"

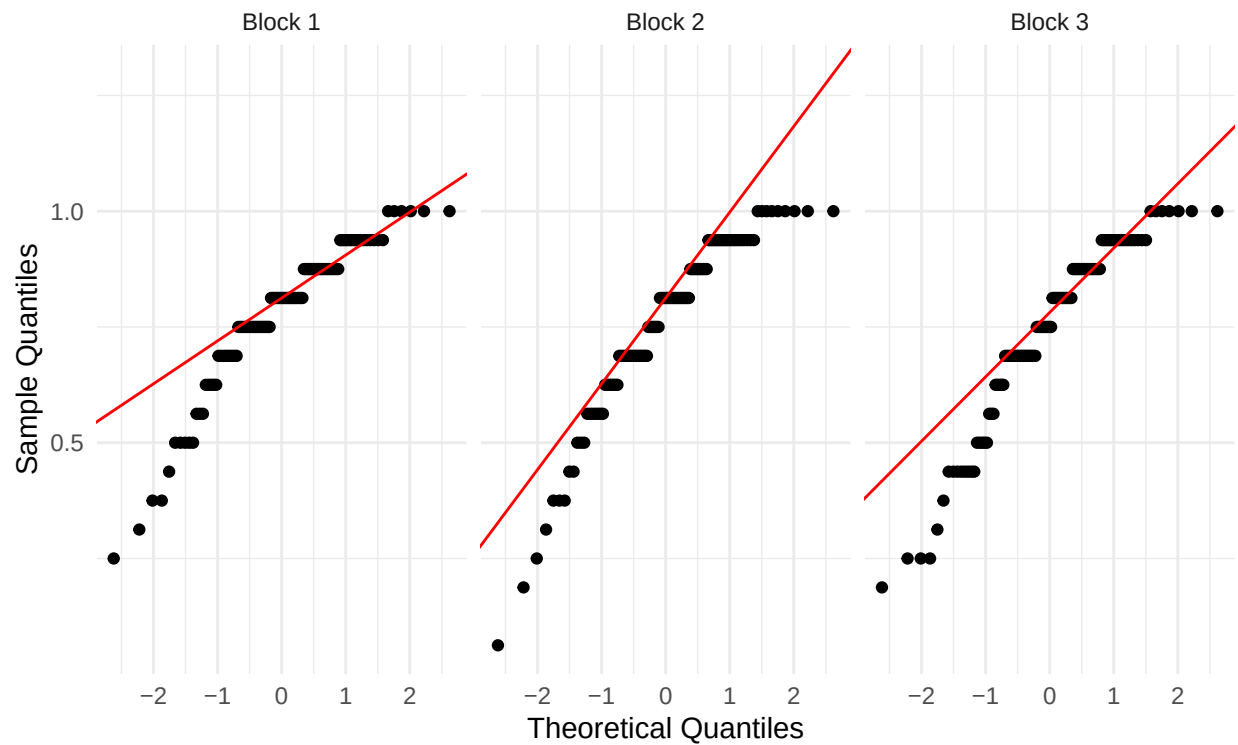
library(ggplot2)

# Q-Q Plot for IR Memorability by Block
ggplot(blocked_aggregated, aes(sample = IR_Memorability)) +
```

```
stat_qq() +
stat_qq_line(color = "red") +
facet_wrap(~ Block_Number, ncol = 3) +
labs(title = "Q-Q Plots: IR Memorability Across Blocks",
      subtitle = "Checking for Normality in Blocks 1, 2, and 3",
      x = "Theoretical Quantiles",
      y = "Sample Quantiles") +
theme_minimal()
```

Q-Q Plots: IR Memorability Across Blocks

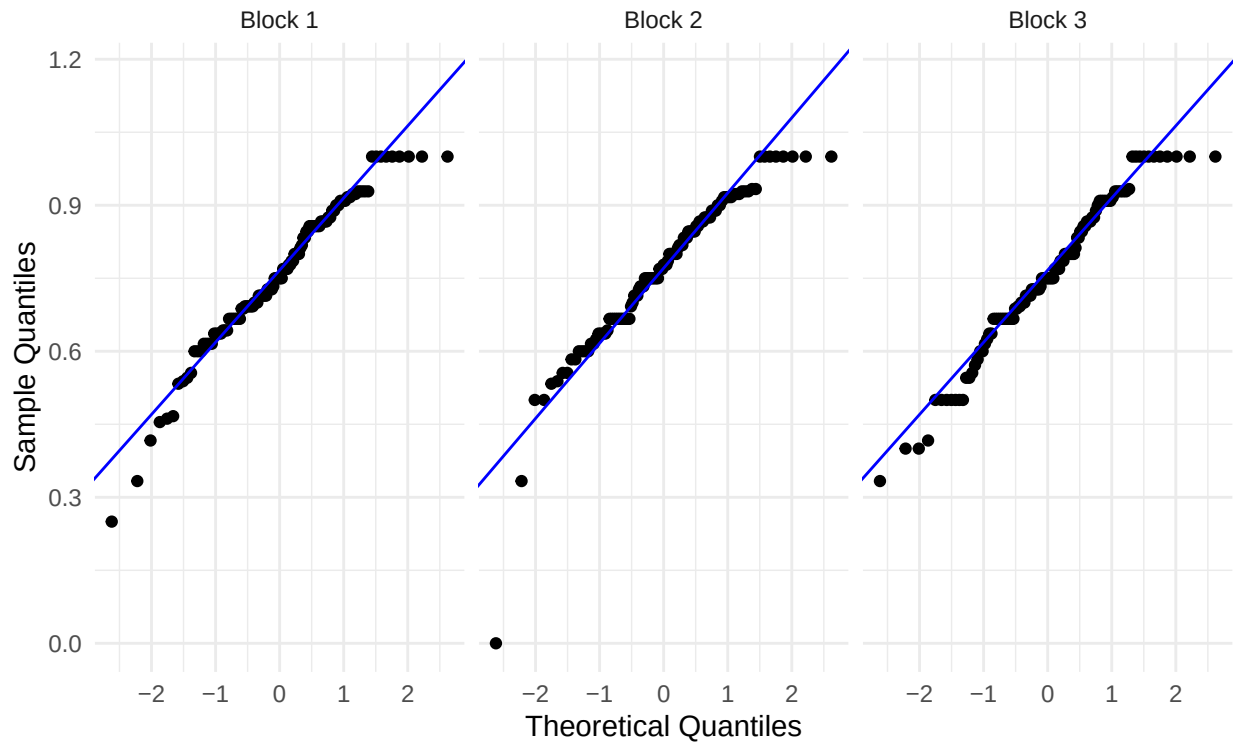
Checking for Normality in Blocks 1, 2, and 3



```
# Q-Q Plot for WR Accuracy by Block
ggplot(blocked_aggregated, aes(sample = WR_Accuracy)) +
  stat_qq() +
  stat_qq_line(color = "blue") +
  facet_wrap(~ Block_Number, ncol = 3) +
  labs(title = "Q-Q Plots: WR Accuracy Across Blocks",
        subtitle = "Checking for Normality in Blocks 1, 2, and 3",
        x = "Theoretical Quantiles",
        y = "Sample Quantiles") +
  theme_minimal()
```

Q-Q Plots: WR Accuracy Across Blocks

Checking for Normality in Blocks 1, 2, and 3



```
# Anderson Darling
library(nortest)

ir_normality_blocks <- blocked_aggregated %>%
  group_by(Block_Number) %>%
  summarise(
    AD_Stat = ad.test(IR_Memorability)$statistic,
    p_value = ad.test(IR_Memorability)$p.value
  )
ir_normality_blocks
```

```
## # A tibble: 3 x 3
##   Block_Number AD_Stat      p_value
##   <fct>      <dbl>      <dbl>
## 1 Block 1      3.21 0.0000000442
## 2 Block 2      2.58 0.00000150
## 3 Block 3      2.77 0.000000516
```

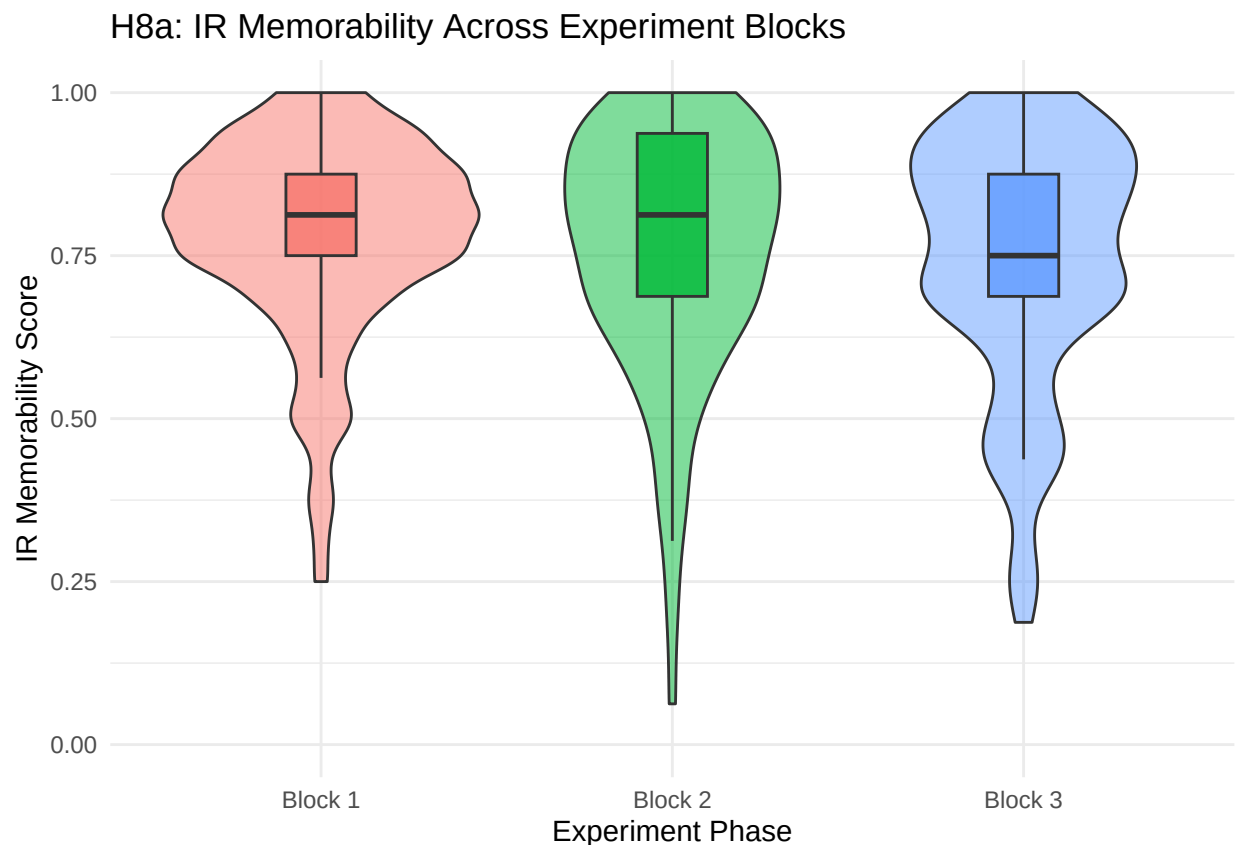
```
wr_normality_blocks <- blocked_aggregated %>%
  group_by(Block_Number) %>%
  summarise(
    AD_Stat = ad.test(WR_Accuracy)$statistic,
    p_value = ad.test(WR_Accuracy)$p.value
  )
wr_normality_blocks
```

```
## # A tibble: 3 x 3
```

```
##   Block_Number AD_Stat p_value
##   <fct>        <dbl>   <dbl>
## 1 Block 1      0.702  0.0653
## 2 Block 2      0.929  0.0178
## 3 Block 3      0.727  0.0565
```

```
# Graph H8a: Violin & Boxplot for IR Memorability across Blocks
```

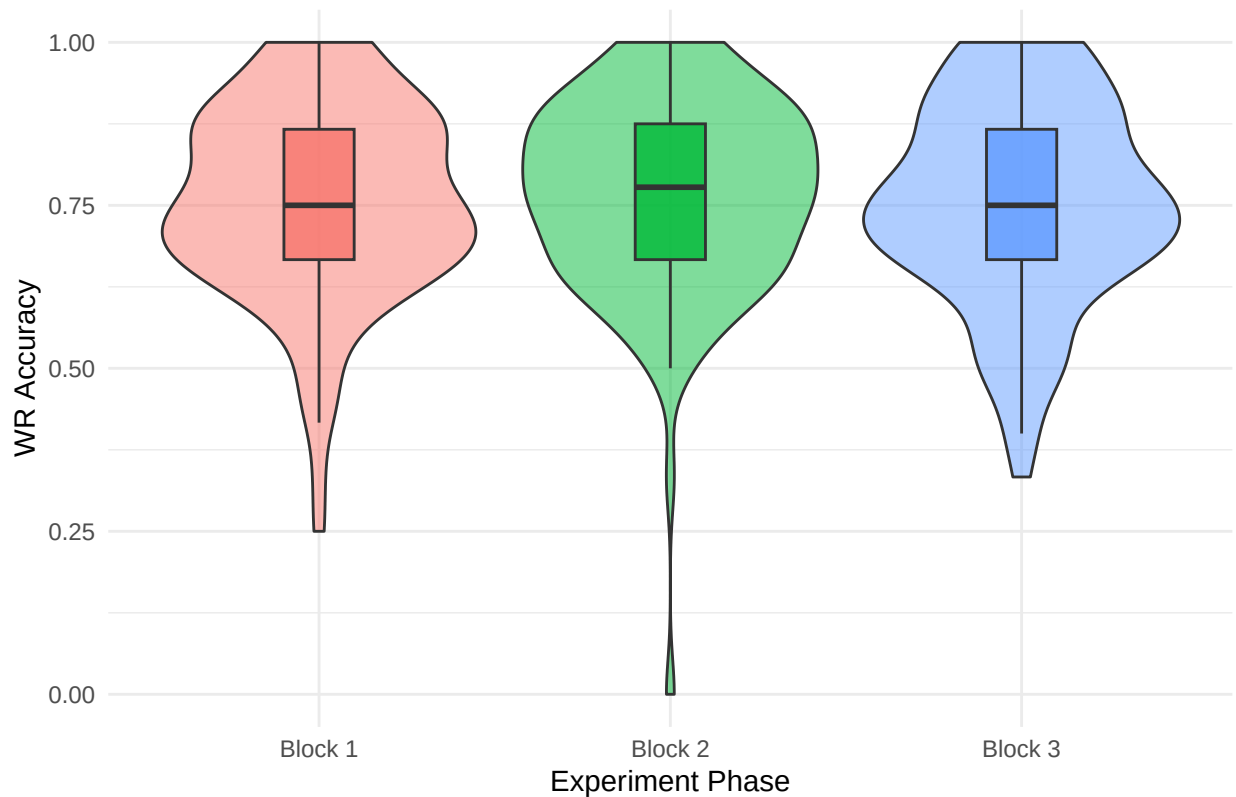
```
ggplot(blocked_aggregated, aes(x = Block_Number, y = IR_Memorability, fill = Block_Number)) +
  geom_violin(alpha = 0.5) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.8) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "H8a: IR Memorability Across Experiment Blocks",
       x = "Experiment Phase",
       y = "IR Memorability Score") +
  theme_minimal() +
  theme(legend.position = "none")
```



```
# Graph H8b: Violin & Boxplot for WR Accuracy across Blocks
```

```
ggplot(blocked_aggregated, aes(x = Block_Number, y = WR_Accuracy, fill = Block_Number)) +
  geom_violin(alpha = 0.5) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.8) +
  labs(title = "H8b: WR Accuracy Across Experiment Blocks",
       x = "Experiment Phase",
       y = "WR Accuracy") +
  theme_minimal() +
  theme(legend.position = "none")
```

H8b: WR Accuracy Across Experiment Blocks



```
# 1. Prepare data for the Friedman test
friedman_data <- blocked_aggregated %>%
  # Step A: Drop NA values
  drop_na(IR_Memorability, WR_Accuracy) %>%
  group_by(participant_ID) %>%
  # Step B: Keep ONLY participants who have exactly 3 unique blocks
  filter(n_distinct(Block_Number) == 3) %>%
  ungroup() %>%
  # Step C: Convert to factors AFTER filtering so empty levels are completely gone
  mutate(
    Block_Number = as.factor(Block_Number),
    participant_ID = as.factor(participant_ID)
  )

# 2. Friedman Test for IR Memorability (H8a)
friedman_ir <- friedman.test(IR_Memorability ~ Block_Number | participant_ID, data = friedman_data)
print("Friedman Test for IR Memorability:")

## [1] "Friedman Test for IR Memorability:"
print(friedman_ir)

##
## Friedman rank sum test
##
## data: IR_Memorability and Block_Number and participant_ID
## Friedman chi-squared = 3.491, df = 2, p-value = 0.1746
```

```

# 3. Friedman Test for WR Accuracy (H8b)
friedman_wr <- friedman.test(WR_Accuracy ~ Block_Number | participant_ID, data = friedman_data)
print("Friedman Test for WR Accuracy:")

## [1] "Friedman Test for WR Accuracy:"
print(friedman_wr)

##
## Friedman rank sum test
##
## data: WR_Accuracy and Block_Number and participant_ID
## Friedman chi-squared = 0.12933, df = 2, p-value = 0.9374
# Count rows in the original aggregated dataset
rows_before <- nrow(blocked_aggregated)

# Count rows in the cleaned Friedman dataset
rows_after <- nrow(friedman_data)

# Calculate how many were dropped
rows_removed <- rows_before - rows_after

# Print the results to the console/document
print(paste("Starting rows:", rows_before))

## [1] "Starting rows: 340"
print(paste("Rows kept for Friedman:", rows_after))

## [1] "Rows kept for Friedman: 339"
print(paste("Total rows removed:", rows_removed))

## [1] "Total rows removed: 1"

```